

Automatización inteligente del flujo de pedidos para un mercado de frutas y verduras

Esp. Ing. Joaquín Manuel Beitia

Maestría en Computación de Borde

Director: Matías Martini (Xepelin)

Jurados:

Jurado 1 (pertenencia)

Jurado 2 (pertenencia)

Jurado 3 (pertenencia)

Ciudad de Rosario, julio de 2026

Resumen

El presente trabajo consiste en el diseño e implementación de un sistema que automatiza la recepción, interpretación y el procesamiento de los pedidos de un mercado de frutas y verduras recibidos a través de mensajería instantánea. El sistema captura pedidos en texto, voz o imagen, e identifica productos, cantidades y unidades mediante modelos de inteligencia artificial. Una vez estructurado el pedido, se imprime en el comercio mediante una impresora térmica conectada a un servidor local. El desarrollo fue realizado para la Tienda de frutas Luján, con el fin de reducir los tiempos operativos, disminuir los errores humanos y mejorar la trazabilidad de la gestión comercial.

La solución se estructura sobre una arquitectura de cómputo distribuida entre el comercio y la nube, en la que las tareas críticas y de baja latencia se ejecutan localmente, mientras que las de mayor demanda de procesamiento se delegan a servicios remotos. El sistema se encuentra operativo en el comercio, donde procesa los pedidos en tiempo real y deja cada pedido listo para su preparación en menos de un minuto desde su recepción. Para su desarrollo se aplicaron conocimientos propios de la computación de borde, tales como la distribución del procesamiento entre el borde y la nube, la comunicación segura entre nodos, la actuación local sobre hardware y el diseño de soluciones de bajo costo.

Agradecimientos

Quiero expresar mi agradecimiento a la Tienda de frutas Luján, y en particular a Marcos y Cristian Cestari, por la confianza depositada en este proyecto y por abrir las puertas de su comercio para el desarrollo y la validación del sistema. Su predisposición y su conocimiento del negocio fueron fundamentales para orientar el trabajo hacia una solución realmente útil.

Agradezco también a mi director, Matías Martini, cuya orientación y experiencia en desarrollo de software e inteligencia artificial aplicada guiaron las decisiones técnicas a lo largo del desarrollo.

Mi reconocimiento se extiende a mis compañeros y colegas, cuya colaboración fue esencial para superar los desafíos surgidos durante la implementación. Por último, agradezco a mis amigos y a mi familia, cuya paciencia y aliento incondicional fueron mi sostén durante este proceso, permitiéndome perseguir mis proyectos y crecer profesionalmente.

Índice general

Resumen	I
1. Introducción general	1
1.1. Mensajería instantánea y gestión de pedidos en comercios minoristas	1
1.2. Contexto y motivación	2
1.3. Estado del arte	3
1.3.1. Soluciones basadas en la nube	3
1.3.2. Plataformas de automatización de mensajería	3
1.3.3. Enfoques de borde e híbridos	4
1.3.4. Comparación y posicionamiento del trabajo	4
1.4. Objetivos del trabajo	4
2. Introducción específica	7
2.1. Dispositivos de borde	7
2.1.1. Servidor local	7
2.1.2. Impresora térmica	8
2.2. Protocolos de comunicación	8
2.2.1. Canal de mensajería	9
2.2.2. Comunicación segura con la nube	9
2.3. Plataformas cloud y servicios	10
2.3.1. Orquestación de flujos de trabajo	10
2.3.2. Infraestructura de despliegue	10
2.3.3. Servicios de inteligencia artificial	10
2.4. Herramientas de almacenamiento y visualización	12
2.4.1. Bases de datos	12
2.4.2. Servidor local y panel de visualización	12
3. Diseño e implementación	13
3.1. Análisis del software	13
4. Ensayos y resultados	15
4.1. Pruebas funcionales del hardware	15
5. Conclusiones	17
5.1. Conclusiones generales	17
5.2. Próximos pasos	17
Bibliografía	19

Índice de figuras

1.1. Esquema conceptual del sistema: capas, componentes y flujo de datos.	2
--	---

Índice de tablas

1.1. Comparación de paradigmas para la automatización de pedidos en el comercio minorista.	4
2.1. Modelos de inteligencia artificial empleados en cada etapa de la interpretación.	11

A mi familia.

Capítulo 1

Introducción general

En el presente capítulo se introduce el área en la que se enmarca el trabajo, se describe el contexto y la motivación que dieron origen al desarrollo, se revisan las soluciones existentes para problemas similares y se enuncian los objetivos perseguidos. El problema abordado es la gestión manual de los pedidos que los comercios reciben por canales de mensajería, y su resolución resulta relevante por el volumen de operaciones que hoy se procesan de esa forma.

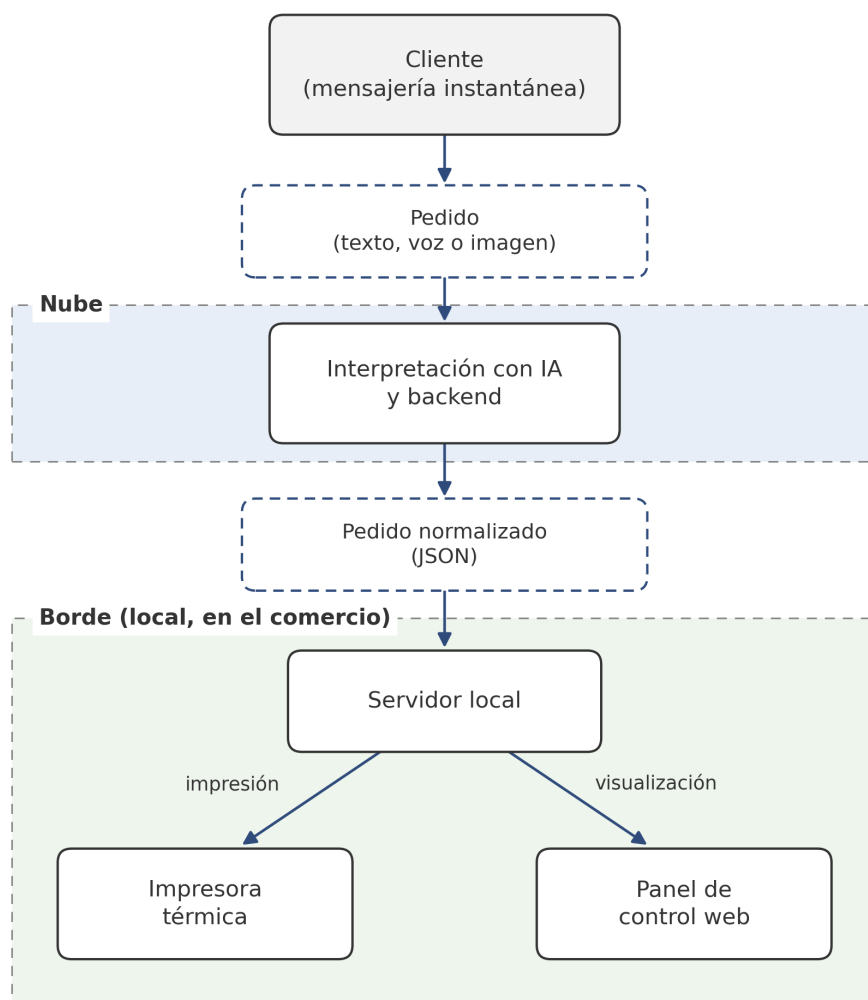
1.1. Mensajería instantánea y gestión de pedidos en comercios minoristas

La creciente adopción de canales de mensajería instantánea como medio de contacto comercial modificó la forma en que los pequeños y medianos comercios reciben los pedidos de sus clientes. En el rubro de los mercados de frutas y verduras, gran parte de las ventas se gestiona a través de aplicaciones de mensajería, donde los pedidos llegan en formatos heterogéneos: texto libre, mensajes de voz, fotografías de listas manuscritas o capturas de pantalla. La interpretación de estos mensajes y su conversión en un pedido preparable constituyen, hoy, una tarea manual realizada por el personal del comercio.

El trabajo se sitúa en la intersección entre la automatización de procesos comerciales y la computación de borde (*edge computing*), entendida esta última como el paradigma en el que parte del procesamiento y la toma de decisiones se ejecutan cerca del lugar donde se generan y se consumen los datos, en lugar de delegarse íntegramente a la nube. En un comercio minorista, el “borde” es el propio local: allí se recibe el pedido, allí debe imprimirse el ticket para su armado y allí ocurre la operación que no puede detenerse ante una caída de conectividad. Este enfoque contrasta con las arquitecturas puramente basadas en la nube, en las que todo el procesamiento se centraliza en servidores remotos.

El sistema desarrollado combina componentes de inteligencia artificial (IA) para la interpretación de los mensajes con una infraestructura de Internet de las cosas (IoT, del inglés *Internet of Things*) para la actuación física en el local, articulados mediante una arquitectura híbrida que distribuye el cómputo entre un servidor local en el comercio y servicios alojados en la nube. El foco de esta memoria está puesto en los aspectos propios de la computación de borde: la distribución del procesamiento entre el borde y la nube, la comunicación segura entre ambos, la actuación local sobre hardware y las restricciones de costo, latencia y disponibilidad características de estos entornos. Los fundamentos de aprendizaje de máquina y aprendizaje profundo se asumen conocidos y no se desarrollan en detalle.

En la figura 1.1 se presenta un esquema conceptual de alto nivel del sistema, en el que el pedido ingresa por el canal de mensajería, se interpreta en la nube y deriva en la impresión local del ticket y en la visualización de la operación. La arquitectura detallada del sistema se describe en el capítulo 3.



Trazo continuo: componentes del sistema.

Trazo punteado: datos que circulan entre los componentes.

FIGURA 1.1. Esquema conceptual del sistema: capas, componentes y flujo de datos.

1.2. Contexto y motivación

El trabajo se desarrolló para la Tienda de frutas Luján, un comercio minorista de frutas y verduras que recibe sus pedidos principalmente a través de mensajería instantánea. En la operación actual, un integrante del negocio lee cada mensaje entrante, interpreta su contenido (que puede llegar como texto, audio o imagen), normaliza los nombres y las cantidades de los productos y transcribe el pedido a un formato que permita su preparación y despacho.

Este proceso manual presenta varias limitaciones. En primer lugar, consume tiempo del personal que podría destinarse a la atención y a la preparación de los pedidos, y su capacidad se satura en los horarios de mayor demanda. En segundo lugar, la transcripción manual es propensa a errores de interpretación, especialmente ante mensajes de voz extensos, listas manuscritas poco legibles o denominaciones ambiguas de un mismo producto. En tercer lugar, al no existir un registro estructurado de los pedidos, se pierde trazabilidad: resulta difícil reconstruir el historial de un cliente, auditar un pedido puntual o extraer información agregada para la gestión comercial.

La motivación del desarrollo es, entonces, doble. Por un lado, se busca reducir los tiempos operativos y los errores humanos automatizando la recepción, la interpretación y la impresión de los pedidos. Por el otro, se procura hacerlo con una solución accesible y de bajo costo, adecuada a la realidad de un comercio que no dispone de infraestructura tecnológica avanzada ni de personal técnico especializado. Esta última condición es la que justifica el enfoque de computación de borde: procesar y actuar localmente permite que la operación crítica (la impresión del ticket para armar el pedido) siga funcionando con baja latencia y sin depender por completo de la disponibilidad de servicios remotos.

1.3. Estado del arte

La automatización de la recepción de pedidos en el comercio minorista puede abordarse desde distintos paradigmas, que se diferencian principalmente por dónde se ejecuta el procesamiento y por el grado de adaptación al canal de mensajería informal. A continuación, se revisan las alternativas existentes y se las contrasta con el enfoque adoptado en este trabajo.

1.3.1. Soluciones basadas en la nube

Una parte importante de las soluciones comerciales de gestión de pedidos y punto de venta se ofrece bajo el modelo de software como servicio, con todo el procesamiento centralizado en la nube. Entre las plataformas más conocidas se encuentran Shopify [1] y, en el mercado local, Tiendanube [2], que ofrecen soluciones de punto de venta y gestión de pedidos. Estas plataformas ofrecen paneles de administración, reportes y, en algunos casos, integración con canales de mensajería. Su principal ventaja es que no requieren infraestructura local más allá de un dispositivo con conexión a Internet. Sus limitaciones, en el contexto de este trabajo, son la dependencia total de la conectividad para operar, los costos recurrentes de suscripción y la escasa capacidad de personalización frente a un flujo de pedidos informal y no estructurado como el que se recibe por mensajería.

1.3.2. Plataformas de automatización de mensajería

Existen plataformas y herramientas de automatización de flujos de trabajo que permiten conectar canales de mensajería con otros servicios mediante reglas o disparadores, como n8n [3] o Zapier [4]. Estas soluciones habilitan respuestas automáticas y la orquestación de tareas, pero por sí solas no resuelven la interpretación semántica de un pedido expresado en lenguaje natural, en audio o en una

imagen. Para ello deben combinarse con modelos de procesamiento de lenguaje, reconocimiento de voz y reconocimiento óptico de caracteres (OCR, del inglés *Optical Character Recognition*).

1.3.3. Enfoques de borde e híbridos

Frente a los esquemas puramente centralizados, la computación de borde propone ejecutar parte del procesamiento cerca del origen de los datos [5, 6]. En el comercio minorista, este enfoque resulta atractivo cuando existe una actuación física local, como la impresión de un ticket, que debe ocurrir con baja latencia y tolerancia a fallos de conectividad. Las arquitecturas híbridas, que combinan un nodo local con servicios en la nube, permiten equilibrar la capacidad de cómputo de la nube con la inmediatez y la resiliencia del procesamiento local [7].

1.3.4. Comparación y posicionamiento del trabajo

En la tabla 1.1 se comparan los tres paradigmas descritos según los criterios más relevantes para el caso de uso abordado. El trabajo desarrollado se posiciona en el enfoque híbrido: delega en la nube las tareas de mayor demanda de cómputo (la interpretación de los mensajes mediante modelos de IA) y ejecuta en el borde la actuación crítica y de baja latencia (la impresión local del pedido), con lo que se busca un compromiso entre costo, personalización y disponibilidad adecuado a un comercio de baja complejidad tecnológica.

TABLA 1.1. Comparación de paradigmas para la automatización de pedidos en el comercio minorista.

Criterio	Nube pura	Automatización de mensajería	Enfoque híbrido (este trabajo)
Procesamiento local	No	Parcial	Sí (actuación local)
Dependencia de conectividad	Alta	Alta	Media
Personalización del flujo	Baja	Media	Alta
Costo de infraestructura	Bajo	Bajo	Bajo/Medio
Trazabilidad estructurada	Variable	Baja	Alta

1.4. Objetivos del trabajo

El objetivo general del trabajo fue diseñar e implementar un sistema de automatización de pedidos. El sistema abarca la recepción, la interpretación y el procesamiento de los pedidos de un mercado de frutas y verduras recibidos a través de mensajería. De este modo se buscó reducir los tiempos operativos, minimizar los errores humanos y mejorar la trazabilidad de la operación comercial.

Para alcanzarlo, se definieron los siguientes objetivos específicos, formulados de manera concreta y verificable:

- Capturar de forma automática los pedidos entrantes por el canal de mensajería, en sus distintos formatos: texto, voz e imagen.
- Interpretar el contenido de cada pedido mediante modelos de inteligencia artificial, identificando productos, cantidades y unidades, y normalizando denominaciones equivalentes de un mismo producto.
- Generar, a partir de esa interpretación, un pedido en formato estructurado y persistirlo de manera que quede registrada la trazabilidad completa, desde la recepción hasta la impresión.
- Imprimir automáticamente el pedido en una impresora térmica conectada a un servidor local, garantizando la actuación en el borde con baja latencia.
- Establecer una comunicación segura entre el servidor local y los servicios en la nube.
- Proveer un panel web que permita visualizar los pedidos en tiempo real, el historial y métricas básicas de operación.
- Validar el sistema en condiciones reales de operación en el comercio.

El alcance del trabajo se limita al flujo de recepción, interpretación, impresión y visualización de pedidos. Las tecnologías y componentes de terceros empleados en la construcción del sistema se describen en el capítulo [2](#), mientras que el diseño y la implementación de la arquitectura se detallan en el capítulo [3](#).

Capítulo 2

Introducción específica

En el presente capítulo se describen las tecnologías, plataformas, servicios y componentes de terceros utilizados como base para el desarrollo del sistema. Se presentan los dispositivos de borde, los protocolos de comunicación, las plataformas en la nube, los servicios de inteligencia artificial, las herramientas de almacenamiento y los recursos de visualización empleados en la solución.

2.1. Dispositivos de borde

El nodo de borde del sistema está constituido por un servidor local instalado en el comercio y por una impresora térmica conectada a él. A diferencia de un sistema de Internet de las cosas tradicional, no se emplean microcontroladores ni sensores. El dispositivo de borde es un equipo de cómputo de propósito general que ejecuta los servicios locales y comanda la impresión de los pedidos.

2.1.1. Servidor local

Como servidor local se utiliza una computadora con sistema operativo Windows Server 2019 [8], propiedad del cliente y preexistente en el local. Sobre este equipo se ejecutan el servidor de impresión y el cliente del túnel seguro hacia la nube.

Se evaluaron dos alternativas para el nodo de borde. La primera consistió en incorporar una computadora de placa única (SBC, del inglés *Single Board Computer*) dedicada, como una Raspberry Pi [9]. La segunda consistió en reutilizar el equipo ya disponible en el comercio. Se optó por la segunda opción a partir de tres criterios.

El primer criterio fue el costo. El trabajo se planteó como una solución económica para un comercio pequeño. La reutilización del hardware disponible elimina la inversión marginal en equipamiento. El segundo criterio fue la operación. El equipo se encuentra en el local, permanece encendido durante el horario comercial y resulta conocido por el personal. Esto simplifica la instalación y el mantenimiento posterior. El tercer criterio fue la compatibilidad. La impresora térmica seleccionada provee controladores para Windows, de modo que la integración no requirió desarrollos adicionales de bajo nivel.

Como contrapartida, un equipo de propósito general consume más energía que una SBC dedicada y puede verse condicionado por otros usos dentro del comercio. Esta limitación se consideró aceptable frente a la simplicidad operativa obtenida.

También se descartó el uso de un microcontrolador. Un dispositivo de este tipo puede ejecutar funciones simples, como un servidor web básico o la comunicación con periféricos. Sin embargo, el conjunto de responsabilidades del nodo de borde (un servidor web local, el cliente del túnel seguro y la administración de la cola de impresión) se resuelve de manera más conveniente sobre un sistema operativo completo.

2.1.2. Impresora térmica

Para la impresión de los tickets se utiliza una impresora térmica Hasar P-HAS-181 [10], un modelo de tipo comandera del fabricante argentino Grupo Hasar, orientado al punto de venta. La impresora imprime por cabezal térmico directo sobre rollos de papel de 80 mm de ancho, con corte automático de papel y resolución de 203 dpi (del inglés, *dots per inch*, puntos por pulgada). Admite la impresión de códigos de barras y códigos QR (del inglés, *Quick Response*). Dispone de tres interfaces de conexión (puerto serie, USB y Ethernet) y de controladores para Windows. Sus comandos de impresión resultan compatibles con ESC/POS [11], el conjunto de instrucciones definido por Epson que constituye el estándar de facto del sector.

En la instalación actual, la impresora se conecta al servidor local por USB. La disponibilidad de una interfaz Ethernet fue, sin embargo, un criterio de selección. El equipo puede requerir un cambio de ubicación dentro del comercio, por ejemplo hacia el sector de armado de pedidos. La conexión por red permitiría ese traslado sin dependencia de la proximidad física con el servidor.

La elección de este modelo se fundamentó en cuatro factores. Primero, la impresión térmica es el estándar del comercio minorista por su velocidad y su bajo costo operativo, dado que no utiliza tinta ni cartuchos. Segundo, la compatibilidad con ESC/POS permite comandar el equipo desde bibliotecas de software ampliamente disponibles, sin dependencia de un protocolo propietario. Tercero, la disponibilidad de controladores para Windows garantiza la integración directa con el servidor local. Cuarto, se trata de un fabricante nacional con presencia establecida en el mercado argentino, lo que asegura la provisión local de insumos, repuestos y servicio técnico. Este último criterio resulta relevante para un comercio sin personal técnico propio.

2.2. Protocolos de comunicación

La comunicación del sistema no se apoya en los protocolos habituales de las redes de sensores, como MQTT o LoRaWAN, sino en protocolos y servicios de la web. El intercambio de mensajes entre los distintos componentes se realiza mediante HTTP (del inglés, *Hypertext Transfer Protocol*), un protocolo de nivel de aplicación basado en peticiones y respuestas, definido en el estándar RFC 9110 [12]. Las interfaces expuestas siguen el estilo arquitectónico REST (del inglés, *Representational State Transfer*), formulado por Fielding [13], que organiza la comunicación alrededor de recursos identificados por direcciones y operaciones uniformes sobre ellos.

Esta elección responde a que todos los servicios involucrados exponen sus funcionalidades a través de interfaces de programación de aplicaciones (API, del inglés

Application Programming Interface) basadas en HTTP. Para los eventos asincrónicos, como la llegada de un mensaje, se emplea el mecanismo de notificaciones automáticas (*webhooks*), incorporado como elemento de primer nivel en la versión 3.1 de la especificación OpenAPI [14]. Este mecanismo invierte el sentido habitual de la comunicación: el servicio que origina el evento emite una petición hacia una dirección configurada por el sistema receptor, sin que este último deba consultar de forma periódica por novedades.

2.2.1. Canal de mensajería

La vinculación con WhatsApp se realiza mediante Evolution API [15], una pasarela de mensajería de código abierto que expone las funciones de la plataforma a través de una interfaz REST y de notificaciones automáticas. Evolution API admite dos modos de conexión. El primero se basa en el protocolo de WhatsApp Web, mediante la biblioteca Baileys, y no requiere verificación comercial ante Meta. El segundo se basa en la API oficial WhatsApp Business Cloud [16]. En este trabajo se utiliza el primer modo. Ante cada mensaje entrante, de texto, voz o imagen, la pasarela emite una notificación con el contenido o con las referencias para descargarlo.

La elección de Evolution API para la etapa inicial del proyecto se fundamentó en tres razones: su costo nulo, la rapidez de puesta en marcha (no requiere el proceso de verificación comercial ni la aprobación de plantillas de la API oficial) y su despliegue autoalojado, coherente con la infraestructura contenerizada del sistema. Su principal limitación reside en que el modo basado en WhatsApp Web constituye un mecanismo no oficial, con posibles restricciones de estabilidad y de cumplimiento frente a la alternativa provista por Meta. Por este motivo, el sistema se diseñó de manera compatible con la API de WhatsApp Business y se encuentra prevista la migración a la vía oficial. Esta compatibilidad se estableció como requerimiento de interoperabilidad desde la planificación del proyecto, de modo que el cambio no implica modificaciones estructurales en los flujos de procesamiento.

2.2.2. Comunicación segura con la nube

Para vincular los servicios en la nube con el servidor local sin apertura de puertos en la red del comercio, se emplea un túnel seguro provisto por Cloudflare [17]. Un cliente ligero, ejecutado en el servidor local, establece una conexión cifrada saliente hacia la infraestructura del proveedor. A través de esa conexión, los servicios remotos alcanzan al nodo de borde.

Las alternativas consideradas presentaban desventajas claras para este escenario. La exposición directa del servidor mediante la apertura de puertos en el enrutador del local requiere una dirección IP pública o un servicio de DNS dinámico (del inglés, *Domain Name System*), exige configurar el equipamiento de red del comercio y deja un servicio a la escucha de conexiones entrantes desde Internet, lo que amplía la superficie de ataque. Una red privada virtual (VPN, del inglés *Virtual Private Network*) entre la nube y el local resuelve la seguridad, pero agrega administración de claves y configuración en ambos extremos. El túnel de Cloudflare, en cambio, no requiere configuración alguna del enrutador, ya que la conexión

es saliente. Además, cifra el tráfico de extremo a extremo y mantiene la red interna del negocio sin exposición entrante. Todo ello con una operación mínima, condición determinante en un local sin personal técnico.

2.3. Plataformas cloud y servicios

2.3.1. Orquestación de flujos de trabajo

La orquestación de las tareas de recepción, interpretación y estructuración de los pedidos se realiza con n8n [3], una plataforma de automatización de flujos de trabajo de código abierto. En n8n, la lógica se construye mediante la conexión de nodos configurables (disparadores, transformaciones e integraciones con servicios externos) en flujos visuales. Este enfoque permite integrar servicios heterogéneos sin desarrollar desde cero la lógica de integración. La plataforma incorpora además agentes de inteligencia artificial (*AI Agents*), que encadenan modelos de lenguaje con herramientas y fuentes de datos. Sobre esa capacidad se apoya la interpretación de los pedidos.

Frente a alternativas comerciales de automatización en la nube, como Zapier [4], n8n presentó dos ventajas decisivas para este trabajo. La primera es la posibilidad de autoalojamiento en la propia infraestructura, sin costo por tarea ejecutada, un factor crítico dado el volumen diario de mensajes. La segunda es que los datos de los clientes permanecen dentro del entorno controlado por el sistema, sin atravesar una plataforma de terceros adicional.

2.3.2. Infraestructura de despliegue

Los servicios en la nube se alojan en un servidor privado virtual (VPS, del inglés *Virtual Private Server*) del proveedor Hostinger [18], en su plan KVM 4. Este plan provee 4 núcleos virtuales de procesamiento, 16 GB de memoria RAM, 200 GB de almacenamiento NVMe (del inglés, *Non-Volatile Memory Express*) y 16 TB de transferencia mensual. El dimensionamiento respondió a la carga concurrente del sistema. Sobre el mismo servidor conviven la pasarela de mensajería, el motor de flujos, las bases de datos y el panel web. El almacenamiento NVMe favorece la baja latencia de las operaciones sobre la base de datos.

Todos los servicios se despliegan de forma contenerizada mediante Docker [19]. La contenerización empaqueta cada servicio junto con sus dependencias en unidades aisladas y reproducibles, lo que aporta tres beneficios concretos. El despliegue completo del sistema se describe en archivos versionables. El entorno puede reproducirse en otro servidor, o en un equipo local de pruebas, sin diferencias de configuración. Cada componente puede actualizarse o reiniciarse de manera independiente, sin afectar al resto.

2.3.3. Servicios de inteligencia artificial

La interpretación de los pedidos requiere resolver tres tareas diferenciadas: convertir los mensajes de voz en texto, leer las listas manuscritas enviadas como fotografía y extraer de un texto informal los productos, las cantidades y las unidades solicitadas. Cada tarea se resuelve con un modelo de inteligencia artificial consumido como servicio en la nube, provisto por OpenAI [20] o por Anthropic [21].

La transcripción de audio se realiza con el servicio de transcripción de OpenAI, basado en el modelo Whisper [22], un sistema de reconocimiento automático del habla entrenado sobre un corpus extenso y multilingüe. Su característica más relevante para este trabajo es la robustez frente a condiciones adversas de grabación: ruido de fondo, variabilidad de acentos y calidad desigual del micrófono. Los mensajes de voz de los clientes se graban dentro del comercio o en la vía pública, sin control sobre el entorno sonoro, por lo que esa robustez resultó determinante.

La lectura de las imágenes se resuelve con modelos multimodales de visión. Se evaluó como alternativa un motor de reconocimiento óptico de caracteres (OCR, del inglés *Optical Character Recognition*) clásico, que transcribe texto a partir de la forma de los caracteres. Las imágenes recibidas presentan una variedad amplia: listas manuscritas fotografiadas con el teléfono, con caligrafías dispares e iluminación irregular, y también capturas de planillas de cálculo enviadas como imagen. En ese escenario, el OCR tradicional pierde precisión, porque opera sobre cada carácter de manera aislada y sin considerar la disposición del contenido. Los modelos multimodales, en cambio, interpretan la imagen en su contexto, reconocen la estructura tabular y aprovechan el conocimiento del dominio para resolver ambigüedades.

Sobre cada imagen se ejecutan dos modelos de proveedores distintos: GPT-4o de OpenAI y Claude Sonnet de Anthropic. La decisión respondió a un problema detectado durante el desarrollo: ante una misma imagen, ambos modelos producen en ocasiones lecturas discrepantes, en particular con caligrafías difíciles o con planillas de estructura compleja. Las dos lecturas se reconcilian luego en un tercer paso, donde un modelo de lenguaje las compara, corrige los errores detectados y devuelve una transcripción única. Esta redundancia agrega costo y latencia, pero reduce el riesgo de un error silencioso en la etapa más frágil de la interpretación.

La estructuración final se resuelve con un modelo extenso de lenguaje (LLM, del inglés *Large Language Model*) de OpenAI, integrado como agente dentro del flujo. El modelo recibe el texto del pedido, ya sea el original o el obtenido de las etapas anteriores, junto con el catálogo de productos del comercio, y produce una representación estructurada con los productos, las cantidades y las unidades. Esta capacidad de interpretar lenguaje informal, con abreviaturas, errores de tipeo y denominaciones no normalizadas, constituye el aporte central de estos modelos a la solución.

En la tabla 2.1 se resumen los modelos empleados en cada etapa.

TABLA 2.1. Modelos de inteligencia artificial empleados en cada etapa de la interpretación.

Etapa	Modelo	Proveedor
Transcripción de audio	Whisper	OpenAI
Lectura de imagen (primera)	GPT-4o	OpenAI
Lectura de imagen (segunda)	Claude Sonnet	Anthropic
Reconciliación de lecturas	GPT-5.5	OpenAI
Estructuración del pedido	GPT-5.5	OpenAI

La decisión de consumir estos modelos como servicio, en lugar de ejecutarlos sobre la propia infraestructura, define la distribución del cómputo entre el borde y la nube. Los tres modelos exigen recursos de procesamiento muy superiores a

los del nodo de borde, por lo que su ejecución local no resultaba viable con el equipamiento del comercio. Su consumo como servicio mantiene la inversión inicial y la complejidad operativa en el mínimo. El costo es doble: un componente variable por uso y la dependencia de la conectividad para la etapa de interpretación. La actuación crítica del sistema permanece en el borde, de manera que esa dependencia no compromete la operación local de impresión.

2.4. Herramientas de almacenamiento y visualización

2.4.1. Bases de datos

La información de pedidos, clientes y productos se almacena en PostgreSQL [23], una base de datos relacional de código abierto, madura y ampliamente adoptada. Su modelo relacional con integridad referencial resulta adecuado para el dominio del problema, donde los pedidos se vinculan con clientes y con artículos del catálogo. La trazabilidad exigida al sistema, es decir la reconstrucción del recorrido de cada pedido desde el mensaje original hasta la impresión, requiere garantías de consistencia transaccional. Las consultas estructuradas sobre el historial alimentan además las métricas del panel de control.

De manera complementaria se emplea Redis [24], una base de datos en memoria de tipo clave-valor, para la gestión de las sesiones y del estado de los clientes activos durante la recepción de un pedido. Redis ofrece tiempos de acceso muy bajos y expiración automática de claves, características apropiadas para datos de vida corta con alta frecuencia de lectura y escritura. La combinación responde a un criterio de diseño explícito: los datos persistentes y auditables residen solo en la base relacional, mientras que Redis se reserva para información temporal. De este modo, una eventual falla de la capa en memoria no compromete los registros del negocio.

2.4.2. Servidor local y panel de visualización

El servidor de impresión que se ejecuta en el nodo de borde se implementó sobre Flask [25], un entorno de desarrollo web minimalista escrito en el lenguaje Python. Flask permite exponer servicios HTTP con una huella de recursos reducida y sin la complejidad de un servidor de aplicaciones completo. Resulta adecuado para un servicio local de propósito específico, que recibe el pedido estructurado y comanda la impresora sobre el equipo Windows del comercio.

La visualización de los pedidos, del historial y de las métricas de operación se ofrece a través de un panel web alojado en el VPS, accesible desde cualquier navegador moderno. Este enfoque evita la instalación de aplicaciones adicionales en los dispositivos del comercio. El panel se desarrolló con Angular [26], un entorno de desarrollo de aplicaciones web mantenido por Google, junto con la biblioteca de componentes Angular Material. La representación de las métricas se resuelve con Chart.js [27]. La elección de un entorno estructurado, con tipado estático y organización por componentes, respondió a la necesidad de sostener el crecimiento del panel a medida que se incorporan nuevas vistas y funciones de administración.

Capítulo 3

Diseño e implementación

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

3.1. Análisis del software

La idea de esta sección es resaltar los problemas encontrados, los criterios utilizados y la justificación de las decisiones que se hayan tomado.

Se puede agregar código o pseudocódigo dentro de un entorno `lstlisting` con el siguiente código:

```
\begin{lstlisting}[caption= "un epígrafe descriptivo"]
  las líneas de código irían aquí...
\end{lstlisting}
```

A modo de ejemplo, se muestra el fragmento de código [3.1](#):

```
1 #define MAX_SENSOR_NUMBER 3
2 #define MAX_ALARM_NUMBER 6
3 #define MAX_ACTUATOR_NUMBER 6
4
5 uint32_t sensorValue[MAX_SENSOR_NUMBER];
6 FunctionalState alarmControl[MAX_ALARM_NUMBER]; //ENABLE or DISABLE
7 state_t alarmState[MAX_ALARM_NUMBER]; //ON or OFF
8 state_t actuatorState[MAX_ACTUATOR_NUMBER]; //ON or OFF
9
10 void vControl() {
11
12     initGlobalVariables();
13
14     period = 500 ms;
15
16     while(1) {
17
18         ticks = xTaskGetTickCount();
19
20         updateSensors();
21
22         updateAlarms();
23
24         controlActuators();
25
26         vTaskDelayUntil(&ticks, period);
27     }
```

28 }

CÓDIGO 3.1. Pseudocódigo del lazo principal de control.

Capítulo 4

Ensayos y resultados

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

4.1. Pruebas funcionales del hardware

La idea de esta sección es explicar cómo se hicieron los ensayos, qué resultados se obtuvieron y analizarlos.

Capítulo 5

Conclusiones

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

5.1. Conclusiones generales

La idea de esta sección es resaltar cuáles son los principales aportes del trabajo realizado y cómo se podría continuar. Debe ser especialmente breve y concisa. Es buena idea usar un listado para enumerar los logros obtenidos.

En esta sección no se deben incluir ni tablas ni gráficos.

Algunas preguntas que pueden servir para completar este capítulo:

- ¿Cuál es el grado de cumplimiento de los requerimientos?
- ¿Cuán fielmente se pudo seguir la planificación original (cronograma incluido)?
- ¿Se manifestó algunos de los riesgos identificados en la planificación? ¿Fue efectivo el plan de mitigación? ¿Se debió aplicar alguna otra acción no contemplada previamente?
- Si se debieron hacer modificaciones a lo planificado ¿Cuáles fueron las causas y los efectos?
- ¿Qué técnicas resultaron útiles para el desarrollo del proyecto y cuáles no tanto?

5.2. Próximos pasos

Acá se indica cómo se podría continuar el trabajo más adelante.

Bibliografía

- [1] Shopify. *Shopify POS: sistema de punto de venta*. Shopify. URL: <https://www.shopify.com/pos> (visitado 06-07-2026).
- [2] Tiendanube. *¿Qué es Punto de Venta de Tiendanube?* Tiendanube. URL: https://ayuda.tiendanube.com/es_AR/pdv/que-es-punto-de-venta-de-tiendanube (visitado 06-07-2026).
- [3] n8n. *n8n: Workflow Automation*. n8n. URL: <https://n8n.io> (visitado 06-07-2026).
- [4] Zapier. *Zapier: Automation for Business*. Zapier. URL: <https://zapier.com> (visitado 06-07-2026).
- [5] Weisong Shi et al. «Edge Computing: Vision and Challenges». En: *IEEE Internet of Things Journal* 3.5 (oct. de 2016), págs. 637-646. DOI: [10.1109/JIOT.2016.2579198](https://doi.org/10.1109/JIOT.2016.2579198).
- [6] Mahadev Satyanarayanan. «The Emergence of Edge Computing». En: *Computer* 50.1 (ene. de 2017), págs. 30-39.
- [7] Liang Tong, Yong Li y Wei Gao. «A Hierarchical Edge Cloud Architecture for Mobile Computing». En: *IEEE INFOCOM 2016 – The 35th Annual IEEE International Conference on Computer Communications*. San Francisco, CA, USA: IEEE, abr. de 2016, págs. 1-9.
- [8] Microsoft. *Windows Server*. Microsoft. URL: <https://www.microsoft.com/en-us/windows-server> (visitado 20-07-2026).
- [9] Raspberry Pi Ltd. *Raspberry Pi*. Raspberry Pi Ltd. URL: <https://www.raspberrypi.com> (visitado 20-07-2026).
- [10] Compañía Hasar. *Impresora térmica P-HAS-181*. Grupo Hasar. URL: <https://compania.grupohasar.com/producto/p-has-181/> (visitado 13-07-2026).
- [11] Seiko Epson Corporation. *ESC/POS Command Reference for TM Printers*. Epson. URL: https://download4.epson.biz/sec_pubs/pos/reference_en/escpos/index.html (visitado 20-07-2026).
- [12] Roy T. Fielding, Mark Nottingham y Julian Reschke. *HTTP Semantics*. Request for Comments 9110. Internet Engineering Task Force, jun. de 2022. DOI: [10.17487/RFC9110](https://doi.org/10.17487/RFC9110). URL: <https://www.rfc-editor.org/info/rfc9110> (visitado 20-07-2026).
- [13] Roy Thomas Fielding. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doct. University of California, Irvine, 2000.
- [14] OpenAPI Initiative. *OpenAPI Specification v3.1.0*. OpenAPI Initiative. 2021. URL: <https://spec.openapis.org/oas/v3.1.0.html> (visitado 20-07-2026).
- [15] Evolution API. *Evolution API: Open-source WhatsApp Integration API*. Evolution API. URL: <https://github.com/EvolutionAPI/evolution-api> (visitado 13-07-2026).
- [16] Meta. *WhatsApp Business Platform*. Meta Platforms. URL: <https://developers.facebook.com/docs/whatsapp> (visitado 06-07-2026).
- [17] Cloudflare. *Cloudflare Tunnel*. Cloudflare. URL: <https://www.cloudflare.com/products/tunnel/> (visitado 06-07-2026).

- [18] Hostinger. *VPS Hosting*. Hostinger. URL: <https://www.hostinger.com/vps-hosting> (visitado 13-07-2026).
- [19] Docker. *Docker: Accelerated Container Application Development*. Docker, Inc. URL: <https://www.docker.com> (visitado 06-07-2026).
- [20] OpenAI. *OpenAI Platform*. OpenAI. URL: <https://platform.openai.com> (visitado 06-07-2026).
- [21] Anthropic. *Claude*. Anthropic. URL: <https://www.anthropic.com/claude> (visitado 13-07-2026).
- [22] OpenAI. *Whisper: Robust Speech Recognition*. OpenAI. URL: <https://github.com/openai/whisper> (visitado 06-07-2026).
- [23] The PostgreSQL Global Development Group. *PostgreSQL: The World's Most Advanced Open Source Relational Database*. The PostgreSQL Global Development Group. URL: <https://www.postgresql.org> (visitado 06-07-2026).
- [24] Redis. *Redis: The Real-time Data Platform*. Redis. URL: <https://redis.io> (visitado 06-07-2026).
- [25] Pallets. *Flask: Web Development, One Drop at a Time*. Pallets. URL: <https://flask.palletsprojects.com> (visitado 06-07-2026).
- [26] Google. *Angular*. Google. URL: <https://angular.dev> (visitado 20-07-2026).
- [27] Chart.js. *Chart.js: Simple yet flexible JavaScript charting library*. Chart.js. URL: <https://www.chartjs.org> (visitado 20-07-2026).